

SUDHIR TUTORIALS WEBAPP

Complete Backend Technical Manual — API Routes, Database Queries & Execution Flows

1. System Architecture & Tech Stack

Sudhir Tutorials WebApp ka backend Next.js 14 Serverless API Routes par base hai. Real-time PostgreSQL database management ke liye Prisma ORM use hota hai, aur external AI services (Google Gemini, Groq Whisper, ElevenLabs) & Payment Gateways (Razorpay) seamlessly integrated hain.

Layer	Tech Stack	Backend Role & Mechanics
API Framework	Next.js 14 App Router API Routes	Serverless Node.js execution (<code>/api/student/...</code> , <code>/api/admin/...</code> , <code>/api/payment/...</code>).
Database Engine	PostgreSQL (Supabase Cloud)	Relational data storage with pooled transaction connections & direct connection URLs.
ORM Layer	Prisma ORM (v5)	Type-safe database queries (<code>prisma.user.findUnique</code> , <code>prisma.\$transaction</code>), auto-migrations.
Authentication	NextAuth.js + bcryptjs	JWT Bearer tokens, salted hash verification (10 rounds), active token single-device guard.
AI & Audio	Gemini 2.5 Flash, Groq Whisper, ElevenLabs	SSE text streaming, multi-part form audio transcription, buffer chunk TTS audio streaming.
Payments	Razorpay SDK + Crypto HMAC	Order creation, HMAC-SHA256 signature verification, webhook processing.

2. Comprehensive Backend API Endpoint Registry

System me total 84 active API Route handlers (`route.ts`) hain. Important endpoints aur unka backend execution logic below detailed hai:

Endpoint Route Path	HTTP Method	Prisma & Backend Execution Logic
<code>/api/auth/[...nextauth]</code>	POST / GET	NextAuth handler. Validates username/password using bcrypt, fetches user role & activeToken, signs JWT.
<code>/api/auth/active-token-check</code>	GET	Fetches activeToken from DB for current userId. Returns <code>{ invalid: true }</code> if session replaced.
<code>/api/student/guru-ji</code>	POST	Parses text/image/PDF. Extracts PDF via <code>pdf-parse</code> . Queries Prisma profile, streams Gemini response.
<code>/api/student/guru-ji/tts</code>	POST	Tries ElevenLabs -> OpenAI Onyx -> Google Audio Stream. Returns chunked <code>audio/mpeg</code> MP3 buffer stream.
<code>/api/student/guru-ji/transcribe</code>	POST	Receives FormData audio WebM file. Forwards buffer stream to Groq Whisper (<code>whisper-large-v3</code>).
<code>/api/payment/create-order</code>	POST	Authenticates session, validates price, instantiates Razorpay instance & creates Razorpay Order ID.
<code>/api/payment/verify</code>	POST	Calculates HMAC-SHA256 signature using <code>RAZORPAY_KEY_SECRET</code> . Executes <code>prisma.\$transaction</code> .
<code>/api/teacher/attendance</code>	GET / POST	Upserts attendance array in <code>Attendance</code> table using composite unique constraint <code>(batchId, studentId, date)</code> .
<code>/api/student/tests/[id]/submit</code>	POST	Compares user answers against correct keys. Computes score, tab violations & creates <code>OnlineTestSubmission</code> .

/api/admin/payments/receipt	GET	Queries `Payment` model with `User` & `Batch` relations, generates formatted invoice data for printing.
-----------------------------	-----	---

3. Authentication, Security & Single-Device Enforcement Backend

A. NextAuth Authorization Code Flow:

Jab user login form submit karta hai, NextAuth `CredentialsProvider` authorize method execute karta hai: 1. Backend ``prisma.user.findUnique({ where: { username } })`` execute karta hai. 2. Hashed password verify hota hai: ``await bcrypt.compare(credentials.password, user.password)``. 3. Successful check par server a unique UUID ``activeToken`` generate karta hai aur DB me write karta hai: ``prisma.user.update({ where: { id: user.id }, data: { activeToken: newUUID } })``. 4. JWT token me ``id``, ``role``, aur ``activeToken`` encode hokar browser HTTP-Only cookie me store ho jata hai.

B. Single Active Device Enforcement (SessionGuard):

Student portal me active duplicate logins prevent karne ke liye ``SessionGuard`` component periodic background polling (``fetch('/api/auth/active-token-check')``) run karta hai. Endpoint verify karta hai: ``if (dbUser.activeToken !== session.activeToken)`` -> Return status 401 & force ``signOut()``. Iss se agar Student 'B' kisi doosri device par login karega, toh purana Student 'A' automatically logout ho jaega.

4. ST Guru ji Multimodal AI & Voice Backend Pipeline

ST Guru ji backend (``/api/student/guru-ji``) complex prompt engineering, context injection, document parsing, aur fallback voice streaming execute karta hai.

Step-by-Step Code Execution Pipeline:

- **1. Payload Extraction & Parsing:** Request body se `{ prompt, image, pdf, selectedLanguage }` receive hota hai.
- **2. PDF Buffer Extraction:** If ``pdf`` (Base64 string) is present: Backend Base64 ko Node.js ``Buffer.from(pdf, 'base64')`` me convert karta hai aur ``pdf-parse`` module se raw text output extract karke prompt context me attach kar deta hai.
- **3. Database Context Injection:** Backend Prisma ORM execute karta hai: ``prisma.studentProfile.findUnique({ where: { userId } })``. Student ki Class, Board, Name retrieve karke System Prompt dynamic create karta hai: ``You are Sudhir Sir addressing Rahul beta (Class 10th CBSE)...``
- **4. AI Model Invocation & SSE Streaming:** ``@google/generative-ai`` SDK initialized hoti hai with ``gemini-2.5-flash``. Model output ``ReadableStream`` me chunk-by-chunk client ko stream hota hai. Agar API error code 429/500 aaye, backend automatically Groq API (``llama-3.3-70b-versatile``) par fall through karta hai.
- **5. Vector SVG Diagram Generator:** Prompt instruction ensure karta hai ki scientific/mathematical visual requests par inline ``code`` block return ho. Visual container ``#0f172a`` high contrast theme maintain karta hai.
- **6. History Retention:** Response complete hoti hi backend Prisma query call karta hai: ``prisma.doubtHistory.create({ data: { studentId, question, response } })``.

Universal TTS Server Audio Streaming (``/api/student/guru-ji/tts``):

Mobile App WebView (Capacitor) me direct browser Web Speech API issue karti hai. Isko solve karne ke liye hamara backend universal MP3 streaming provider ki tarah work karta hai: 1. Input text ko max 200 characters chunks me trim karta hai. 2. Alternate Google Translate Audio API endpoints (``https://translate.google.com/translate_tts?ie=UTF-8&q;=...``) par HTTP requests send karta hai with browser headers. 3. Sabhi incoming audio buffers ko concatenate karta hai Node.js ``Buffer.concat(chunks)``. 4. Client  standard HTTP response header ``Content-Type: audio/mpeg`` ke sath binary audio stream karta hai. Result: Har device par clear audio playback without any browser alert!

5. E-Commerce & Razorpay Payment Verification Backend

E-Commerce store me course notes aur online test series khareedne ke liye secure cryptographic payment pipeline build ki gayi hai.

Razorpay Order Creation (/api/payment/create-order):

```
Client jab 'Buy Now' click karta hai, backend `prisma.storeItem.findUnique({ where: { id: storeItemId } })` se price verify karta hai (tampering prevent karne ke liye). Phir Razorpay SDK call hota hai: const order = await razorpay.orders.create({ amount: item.price * 100, currency: 'INR', receipt: `rcpt_${Date.now()}` });
```

Cryptographic Signature Verification (/api/payment/verify):

Payment finish hone par Razorpay frontend ko `razorpay\_order\_id`, `razorpay\_payment\_id`, aur `razorpay\_signature` deta hai. Backend inko verify karta hai Node.js `crypto` library se: const generated_signature = crypto.createHmac('sha256', process.env.RAZORPAY_KEY_SECRET).update(order_id + '|' + payment_id).digest('hex'); Agar `generated\_signature` === `razorpay\_signature`, tabhi backend `prisma.\$transaction` run karke `StorePurchase` update karta hai aur course content instant unlock kar deta hai.

6. Database Relational Models (Prisma Schema Summary)

Database total 25 relational models maintain karta hai. Key relational models and foreign key links:

Prisma Model	Key Columns & Types	Foreign Key Relations & Cascades
`User`	id (cuid), username, password, role, activeToken	1-to-1 with `StudentProfile`, `TeacherProfile`. 1-to-Many with `Payment`, `Attendance`.
`StudentProfile`	id, userId, class, board, parentPhone	FK: `userId` -> `User.id` (ON DELETE CASCADE). Belongs to `Batch`.
`Batch`	id, name, courseId, feeAmount	FK: `courseId` -> `Course.id`. Has Many `StudentProfile`, `Schedule`, `Attendance`.
`Payment`	id, studentId, amount, receiptNo, status, month	FK: `studentId` -> `User.id`. Tracks monthly institute fee collection.
`Attendance`	id, studentId, batchId, date, status	FK: `studentId` -> `User.id`, `batchId` -> `Batch.id`. Unique composite constraint `(studentId, batchId, date)`.
`StoreItem`	id, title, price, itemType, contentUrl	Has Many `StorePurchase` records and `OnlineTest` entries.
`OnlineTest`	id, storeItemId, durationMins, totalMarks	FK: `storeItemId` -> `StoreItem.id`. Has Many `OnlineTestQuestion` & `OnlineTestSubmission`.

Architectural Conclusion: Next.js Serverless architecture, Prisma relational integrity, Razorpay cryptographic verifications, aur multi-tier AI/TTS audio fallbacks milkar Sudhir Tutorials WebApp ko ek high-performance, robust aur enterprise-grade platform banate hain.